

LAPORAN PRAKTIKUM DESAIN PEMROGRAMAN WEB

Jobsheet-07: PHP - Form Proccessing

Dosen Pengajar:

Dimas Wahyu Wibowo, ST., MT



Disusun oleh:

Nama : Marta Prama Daniswara

NIM : 244107020205

Kelas : TI-2F

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

MALANG

OKTOBER 2025



Jurusan Teknologi Informasi Politeknik Negeri Malang

Jobsheet-7: PHP - Form Proccessing

Mata Kuliah Desain dan Pemrograman Web

Pengampu: Tim Ajar Desain dan Pemrograman Web

Oktober 2024

Topik

- Konsep Form Proccessing dengan PHP dan JQuery

Tujuan

Mahasiswa diharapkan dapat:

1. Mahasiswa mampu membuat form menggunakan PHP
2. Mahasiswa mampu membuat form menggunakan jQuery

Perhatian

Jobsheet ini harus dikerjakan step-by-step sesuai langkah-langkah praktikum yang sudah diberikan. Soal dapat dijawab langsung di dalam kolom yang disediakan dengan menggunakan PDF Editor.

Pendahuluan

Function isset

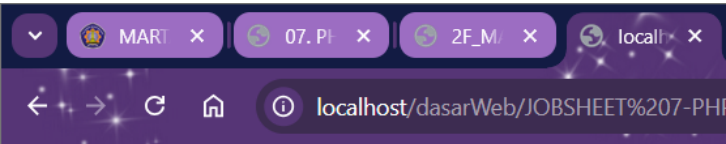
isset() adalah sebuah fungsi dalam PHP yang digunakan untuk memeriksa apakah suatu variabel telah didefinisikan (ada) atau tidak. Fungsi ini mengembalikan **true** jika variabel telah didefinisikan dan memiliki nilai, dan **false** jika variabel tidak ada atau bernilai **null**. Berikut adalah contoh script dan penjelasan minimal mengenai **isset()**:

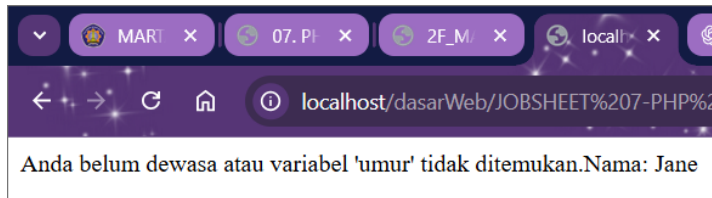
```
$nama = "John";  
if (isset($nama)) {  
    echo "Variabel 'nama' telah didefinisikan.";  
} else {  
    echo "Variabel 'nama' tidak ditemukan atau bernilai null.";  
}
```

Kode di atas memeriksa apakah variabel **\$nama** telah didefinisikan. Jika ya, maka pesan "Variabel 'nama' telah didefinisikan." akan ditampilkan, jika tidak, maka pesan "Variabel 'nama' tidak ditemukan atau bernilai null." akan ditampilkan.

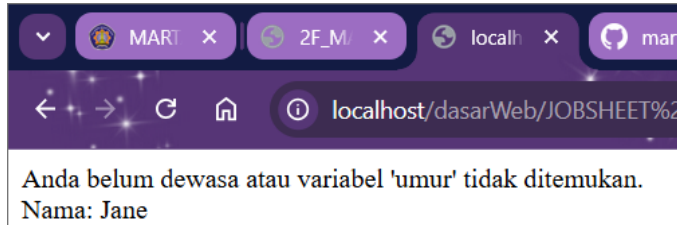
Praktikum 1. Function Isset

Langkah	Keterangan
---------	------------

1	Buatlah satu file baru di dalam direktori dasarWeb, beri nama <code>isset.php</code> .
2	Ketikkan ke dalam file <code>isset.php</code> tersebut kode di bawah ini.
3	<pre> \$umur; if (isset(\$umur) && \$umur >= 18) { echo "Anda sudah dewasa."; } else { echo "Anda belum dewasa atau variabel 'umur' tidak ditemukan."; } </pre>
4	Simpan file tersebut, kemudian buka browser dan jalankan <code>localhost/dasarWeb/isset.php</code>. Apa yang anda pahami dari penggunaan <code>isset</code> pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 1.1) Jawab:  Anda belum dewasa atau variabel 'umur' tidak ditemukan. - <code>isset</code> berfungsi sebagai pengaman agar tidak terjadi error ketika kita mencoba mengakses variabel yang mungkin belum dibuat. - Dengan <code>isset</code>, kita bisa membedakan: • Variabel memang belum ada / tidak dikirimkan dari form. • Variabel ada, tapi nilainya tidak memenuhi syarat tertentu (misalnya umur < 18).
5	Tambahkan isi dari file <code>isset.php</code> tersebut dengan kode di bawah ini. <pre> \$data = array("nama" => "Jane", "usia" => 25); if (isset(\$data["nama"])) { echo "Nama: " . \$data["nama"]; } else { echo "Variabel 'nama' tidak ditemukan dalam array."; } </pre>
6	Simpan file tersebut, kemudian buka browser dan jalankan <code>localhost/dasarWeb/isset.php</code> buat hasil tidak menjadi 1 baris, jadi tampilan dari <code>echo</code> harus terpisah. Apa yang anda pahami dari penggunaan <code>isset</code> pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 1.2) Jawab: - Sebelum perubahan



- Setelah perubahan



- **Solusi:** menambahkan `<br>` pada teks terakhir di baris 7.

```

JOBSHEET 7-PHP FORM > isset.php
1  <?php
2  $umur;
3
4  if (isset($umur) && $umur >= 18) {
5      echo "Anda sudah dewasa.";
6  } else {
7      echo "Anda belum dewasa atau variabel 'umur' tidak ditemukan.<br>";
8  }
9
10 $data = array("nama" => "Jane", "usia" => 25);
11 if (isset($data["nama"])) {
12     echo "Nama: " . $data["nama"];
13 } else {
14     echo "Variabel 'nama' tidak ditemukan dalam array.";
15 }
16 >

```

Function empty

Fungsi **empty()** pada PHP digunakan untuk memeriksa apakah suatu variabel kosong atau tidak terdefinisi. Fungsi ini mengembalikan **true** jika variabel kosong atau tidak terdefinisi, dan **false** jika variabel memiliki nilai atau telah didefinisikan. Berikut adalah penjelasan dan contoh penggunaan **empty()**:

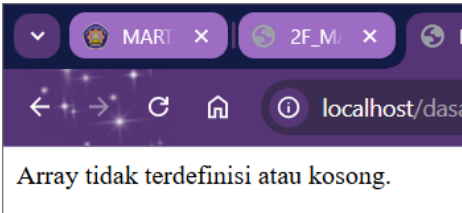
```

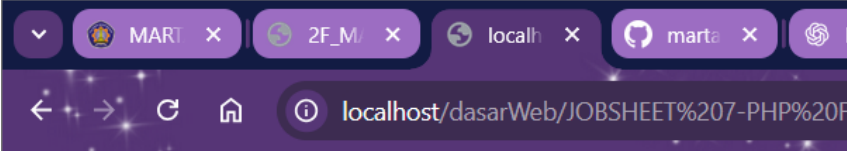
$nama = ""; // Variabel string kosong
if (empty($nama)) {
    echo "Nama tidak terdefinisi atau kosong.";
} else {
    echo "Nama terdefinisi dan tidak kosong.";
}

```

Fungsi **empty()** dapat digunakan untuk memeriksa apakah suatu string kosong atau tidak.

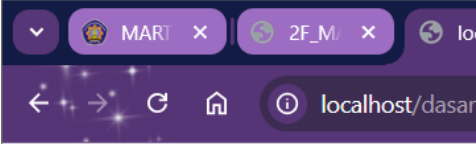
Praktikum 2. Function empty

Langkah	Keterangan
1	Buat satu file baru bernama <code>empty.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>empty.php</code>
2	<pre> \$myArray = array(); // Array kosong if (empty(\$myArray)) { echo "Array tidak terdefinisi atau kosong."; } else { echo "Array terdefinisi dan tidak kosong."; } </pre>
3	Simpan file tersebut, kemudian buka browser dan jalankan <code>localhost/dasarWeb/empty.php</code>. Apa yang anda pahami dari penggunaan <code>empty</code> pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 2.1) Jawab:  Fungsi <code>empty()</code> pada file tersebut dipakai untuk mengecek apakah sebuah variabel sudah berisi nilai yang "ada" atau tidak. • Jika kosong → akan tampil pesan <i>"Array tidak terdefinisi atau kosong."</i> • Jika ada isinya → akan tampil pesan <i>"Array terdefinisi dan tidak kosong."</i>
4	Tambahkan isi dari file <code>empty.php</code> tersebut dengan kode di bawah ini. <pre> if (empty(\$nonExistentVar)) { echo "Variabel tidak terdefinisi atau kosong."; } else { echo "Variabel terdefinisi dan tidak kosong."; } </pre>
5	Simpan file tersebut, kemudian buka browser dan jalankan <code>localhost/dasarWeb/empty.php</code> buat hasil tidak menjadi 1 baris, jadi tampilan dari <code>echo</code> harus terpisah. Apa yang anda pahami dari penggunaan <code>empty</code> pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 2.2) Jawab: - Sebelum perubahan

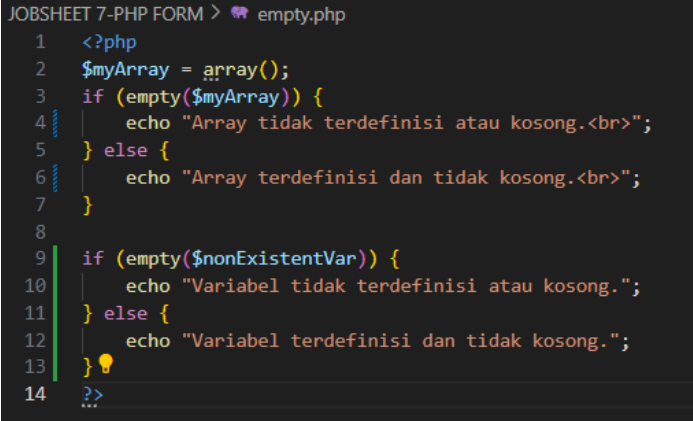


Array tidak terdefinisi atau kosong. Variabel tidak terdefinisi atau kosong.

- Setelah perubahan



Array tidak terdefinisi atau kosong.
Variabel tidak terdefinisi atau kosong.
- Solusi: menambahkan `<br>` di akhir kalimat pada baris 4 dan 6.



```

1  <?php
2  $myArray = array();
3  if (empty($myArray)) {
4      echo "Array tidak terdefinisi atau kosong.<br>";
5  } else {
6      echo "Array terdefinisi dan tidak kosong.<br>";
7  }
8
9  if (empty($nonExistentVar)) {
10     echo "Variabel tidak terdefinisi atau kosong.";
11 } else {
12     echo "Variabel terdefinisi dan tidak kosong.";
13 }
14 ?>

```
- Penjelasan:
 - `empty()` digunakan untuk mengecek apakah variabel kosong atau bahkan belum pernah dibuat.
 - Dengan `empty()`, program lebih aman karena tidak akan error meskipun variabel belum ada.
 - Hasil akan berbeda jika variabel punya nilai → maka dianggap tidak kosong.

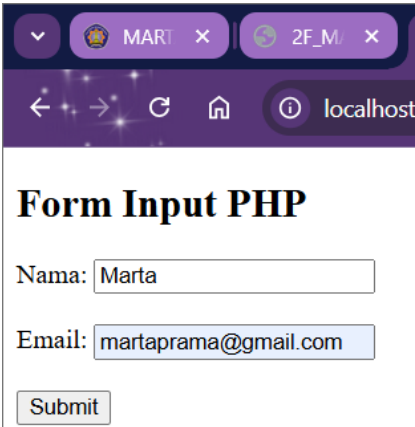
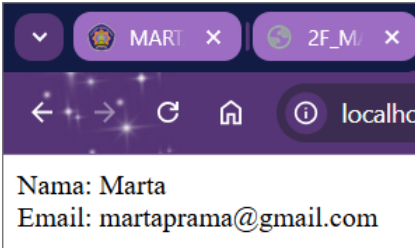
Praktikum Bagian 3 : Form Input PHP

Langka h	Keterangan
1	Buat satu file baru bernama <code>form.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>form.php</code>

2	<pre> <!DOCTYPE html> <html> <head> <title>Form Input PHP</title> </head> <body> <h2>Form Input PHP</h2> <form method="post" action="proses_form.php"> <label for="nama">Nama:</label> <input type="text" name="nama" id="nama" required>

 <label for="email">Email:</label> <input type="email" name="email" id="email" required>

 <input type="submit" name="submit" value="Submit"> </form> </body> </html> </pre>
3	Buat satu file baru bernama proses_form.php di dalam folder dasarWeb. Ketikkan kode pada langkah 2 di dalam proses_form.php <pre> <?php if (\$_SERVER["REQUEST_METHOD"] == "POST") { \$nama = \$_POST["nama"]; \$email = \$_POST["email"]; echo "Nama: " . \$nama . "
"; echo "Email: " . \$email; } ?> </pre>
4	Simpan file tersebut, kemudian buka browser dan jalankan localhost/dasarWeb/proses_form.php. Apa yang anda pahami dari penggunaan form pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 3.1) Jawab: • Form HTML digunakan untuk menangkap input dari user (dalam hal ini Nama dan Email). • Data dikirim ke server menggunakan metode POST, lebih aman karena data tidak terlihat di URL. • File proses_form.php berfungsi untuk menerima dan memproses data yang dikirimkan dari form. • Superglobal \$_POST digunakan untuk mengambil nilai input dari form. • Hasil input ditampilkan kembali dengan echo, sehingga user bisa melihat data yang sudah dikirim.

	- Tampilan awal  Form Input PHP Nama: Marta Email: martaprama@gmail.com Submit - Tampilan setelah di submit  Nama: Marta Email: martaprama@gmail.com
5	Buat satu file baru bernama <code>form_self.php</code> di dalam folder <code>dasarWeb</code>. Ketikkan kode pada langkah 2 di dalam <code>form_self.php</code>

```

<!DOCTYPE html>
<html>
<head>
    <title>Form Input PHP</title>
</head>
<body>
    <h2>Form Input PHP</h2>
    <?php
    // Inisialisasi variabel
    $namaErr = "";
    $nama = "";

    // Cek apakah form sudah disubmit
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        // Validasi nama (contoh: pastikan nama tidak kosong)
        if (empty($_POST["nama"])) {
            $namaErr = "Nama harus diisi!";
        } else {
            $nama = $_POST["nama"];
            echo "Data berhasil disimpan!";
        }
    }
    ?>

    <form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">
        <label for="nama">Nama:</label>
        <input type="text" name="nama" id="nama" value="<?php echo $nama; ?>">
        <span class="error"><?php echo $namaErr; ?></span><br><br>

        <input type="submit" name="submit" value="Submit">
    </form>
</body>
</html>

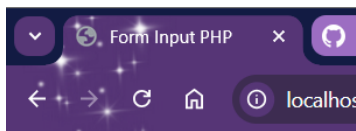
```

6

Simpan file tersebut, kemudian buka browser dan jalankan localhost/dasarWeb/form_self.php. Apa yang anda pahami dari penggunaan form pada file tersebut. Catat di bawah ini pemahaman anda. (soal no 3.2)

Jawab:

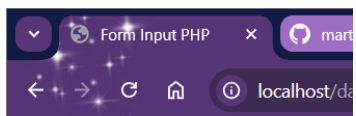
- Tampilan awal



Form Input PHP

Nama:

- Tampilan setelah di submit



Form Input PHP

Data berhasil disimpan!

Nama:

- **Penjelasan:**

- Form digunakan untuk menerima input dari user (dalam hal ini Nama).
- Metode POST dipakai agar data dikirim ke server tanpa terlihat di URL.

- `$_SERVER["REQUEST_METHOD"] == "POST"` berfungsi mengecek apakah form sudah disubmit.
- `empty($_POST["nama"])` digunakan untuk validasi agar field nama tidak boleh kosong.
- Pesan error disimpan di variabel `$namaErr`, lalu ditampilkan kembali di samping input.
- `htmlspecialchars($_SERVER["PHP_SELF"])` dipakai agar form memproses dirinya sendiri, sekaligus mencegah serangan XSS.
- Feedback diberikan langsung kepada user melalui `echo` → baik saat ada error maupun saat data berhasil disimpan.
- Jadi, file tersebut memperlihatkan konsep dasar form input + validasi sederhana di PHP untuk memastikan data yang dikirim user tidak kosong sebelum diproses.

HTML Injection

HTML injection (atau disebut juga "HTML injection attack" atau "client-side injection") adalah serangan keamanan yang terjadi ketika seorang penyerang memasukkan kode HTML atau JavaScript yang berbahaya ke dalam input yang diterima oleh sebuah aplikasi web. Kode berbahaya tersebut akan dieksekusi oleh browser pengguna yang melihat halaman web yang terpengaruh, yang dapat mengakibatkan akses ilegal ke data, perubahan tampilan halaman, atau serangan lainnya.

Seorang penyerang dapat mencoba menyisipkan kode HTML atau JavaScript yang berbahaya ke dalam input yang diterima oleh aplikasi web. Jika aplikasi web tersebut tidak melakukan sanitasi atau escape input dengan benar, maka kode berbahaya tersebut akan dieksekusi oleh browser pengguna, yang dapat menyebabkan masalah keamanan.

Contoh HTML Injection:

Misalnya, kita memiliki formulir pencarian sederhana di situs web yang mencari kata kunci di antara ulasan pengguna:

```
<form action="search.php" method="post">
  <input type="text" name="keyword">
  <input type="submit" value="Search">
</form>
```

Sekarang, jika aplikasi web tidak memvalidasi input dengan benar, seorang penyerang dapat memasukkan input berbahaya seperti ini:

```
<script>alert('You have been hacked!');</script>
```

Jika aplikasi web tidak menghindari atau membersihkan input ini sebelum menampilkannya di halaman hasil pencarian, maka hasilnya akan menjadi seperti ini:

```
<div>Hasil pencarian untuk "<script>alert('You have been hacked!');</script>":</div>
```

Kode JavaScript berbahaya tersebut akan dieksekusi oleh browser pengguna dan akan memunculkan kotak peringatan "You have been hacked!". Ini adalah contoh HTML injection yang sederhana. Sebagai pengembang web, penting untuk selalu melakukan sanitasi dan validasi input dari pengguna, serta menghindari menampilkan input pengguna langsung di halaman web tanpa escaping atau sanitasi yang tepat untuk menghindari serangan semacam ini.

Cara Mengangani HTML Injection

Cara menangani HTML injection adalah dengan memvalidasi, menyaring, dan menghindari menampilkan input pengguna tanpa escaping yang tepat. Berikut adalah beberapa langkah untuk melindungi aplikasi Anda dari HTML injection:

1. **Validasi Input:** Validasi input dari pengguna untuk memastikan bahwa hanya data yang valid yang diterima. Anda bisa menggunakan fungsi seperti `filter_input()` atau `filter_var()` untuk melakukan validasi.
2. **Saring Input:** Saring input pengguna untuk menghapus atau menggantikan karakter yang berpotensi berbahaya, seperti `<`, `>`, `&`, dan lainnya. Anda bisa menggunakan fungsi seperti `htmlspecialchars()` atau `strip_tags()` untuk ini.
3. **Parameterized Statements (Query):** Jika Anda menghasilkan SQL query dengan input pengguna, gunakan parameterized statements atau prepared statements untuk mencegah SQL injection, yang bisa menjadi bentuk serangan yang mirip dengan HTML injection.
4. **Content Security Policy (CSP):** Terapkan Content Security Policy (CSP) pada header HTTP Anda untuk mengontrol sumber daya yang dapat digunakan dalam halaman web Anda.
5. **Escape Output:** Ketika Anda menampilkan data di halaman web, pastikan Anda menghindari injeksi JavaScript dengan menggunakan `htmlspecialchars()` atau metode serupa.

Berikut contoh sederhana dalam PHP untuk mengatasi HTML injection:

```
// Menggunakan filter_input untuk validasi input
$user_input = filter_input(INPUT_POST, 'user_input', FILTER_SANITIZE_STRING);

// Menggunakan htmlspecialchars untuk menghindari HTML injection saat menampilkan data
echo '<div>' . htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8') . '</div>';
```

Dengan menggunakan langkah-langkah ini, Anda dapat memitigasi risiko HTML injection dalam aplikasi web Anda. Selalu penting untuk memvalidasi input, membersihkan data yang masuk, dan menghindari menampilkan data pengguna tanpa escaping yang benar.

Praktikum 4 : HTML Injection

Langkah-langkah Praktikum :

Langkah	Keterangan
1	Buat satu file baru bernama <code>html_aman.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>html_aman.php</code>
2	<pre>\$input = \$_POST['input']; \$input = htmlspecialchars(\$input, ENT_QUOTES, 'UTF-8');</pre>
3	Tambahkan script lainnya supaya langkah 2 bisa berjalan normal. Lalu simpan file , kemudian buka browser dan jalankan/refresh <code>localhost/dasarWeb/html_aman.php</code>
4	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 4.1) Jawab:

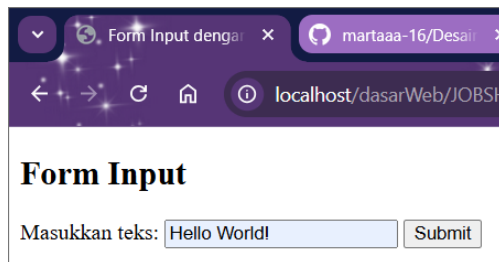
- Script tambahan

```

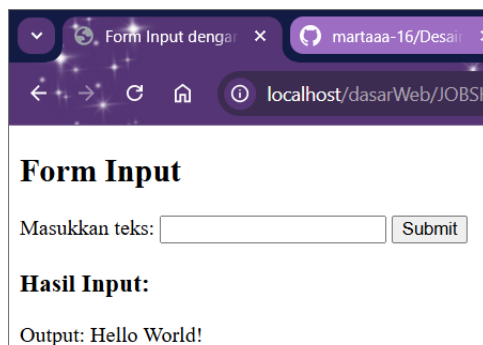
JOBSHEET 7-PHP FORM > html_injection.php
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Form Input dengan htmlspecialchars</title>
5  </head>
6  <body>
7    <h2>Form Input</h2>
8    <form method="post" action="">
9      <label for="input">Masukkan teks:</label>
10     <input type="text" name="input" id="input" required>
11     <input type="submit" value="Submit">
12   </form>
13
14   <?php
15   if ($_SERVER["REQUEST_METHOD"] == "POST") {
16     $input = $_POST['input'];
17     // Melindungi dari XSS dengan htmlspecialchars
18     $input = htmlspecialchars(string: $input, flags: ENT_QUOTES, encoding: 'UTF-8');
19
20     echo "<h3>Hasil Input:</h3>";
21     echo "Output: " . $input;
22   }
23   ?>
24 </body>
25 </html>

```

- Tampilan awal

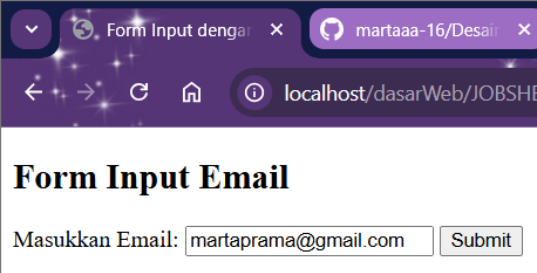
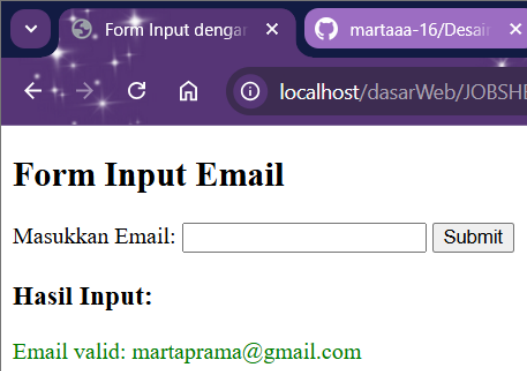


- Tampilan setelah submit



- Penjelasan:

- Sekarang ada form HTML untuk mengirim data ke server dengan metode POST.
- Input yang dimasukkan user diproses menggunakan `htmlspecialchars()`, sehingga karakter khusus seperti `<`, `>`, `'`, `"` tidak dijalankan sebagai kode HTML/JavaScript (aman dari XSS).
- Hasil input ditampilkan kembali di browser dengan aman.
- Program menjadi lengkap dan interaktif: user bisa mengetik di form, submit, lalu langsung melihat hasil inputnya.

6	<pre>// Memeriksa apakah input adalah email yang valid \$email = \$_POST['email']; if (filter_var(\$email, FILTER_VALIDATE_EMAIL)) { // Lanjutkan dengan pengolahan email yang aman } else { // Tangani input yang tidak valid }</pre>
7	Lengkapi kode program di atas sehingga hasilnya rapi. Simpan file , kemudian buka browser dan jalankan/refresh localhost/dasarWeb/html_aman.php
8	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 4.2) Jawab: - Tampilan awal  - Tampilan setelah submit  - Penjelasan: Dari penambahan kode, input pengguna lebih aman karena difilter dengan htmlspecialchars dan divalidasi menggunakan filter_var untuk memastikan format email benar. Program juga memberi umpan balik langsung apakah input valid atau tidak.

Regular Expression (Regex)

Regex adalah alat yang kuat untuk mencari, mencocokkan, atau memanipulasi teks berdasarkan pola tertentu. Anda dapat menggunakannya untuk berbagai tujuan seperti validasi input, pencarian teks, penggantian teks, dan banyak lagi.

Basic Regex Patterns:

- **Karakter Tunggal:** Karakter apa pun akan cocok dengan dirinya sendiri dalam Regex, kecuali karakter khusus yang harus dihindari dengan karakter pelarian \.
 - o Contoh: Pola /a/ akan cocok dengan huruf "a" dalam teks.
- **Karakter Spesifik:** Anda dapat mencocokkan karakter tertentu dengan menyebutkannya dalam pola.
 - o Contoh: Pola /hello/ akan cocok dengan teks "hello" dalam teks.
- **Karakter Khusus:** Beberapa karakter khusus dalam Regex harus dihindari dengan karakter pelarian (\) jika Anda ingin mencocokkannya secara harfiah. Contoh karakter khusus: ., *, +, ?, |, [,], (,), {, }, ^, \$, \.
- **Karakter Set:** Anda dapat mencocokkan karakter dari satu set karakter dengan menggunakan []. Misalnya, /[aeiou]/ akan mencocokkan salah satu huruf vokal.
- **Rentang Karakter:** Anda dapat menggunakan - dalam karakter set untuk menentukan rentang karakter. Misalnya, /[a-z]/ akan mencocokkan huruf kecil apa pun.
- **Kuantifier:** Anda dapat mengatur berapa kali karakter atau kelompok karakter sebelumnya harus muncul. Contoh kuantifier:
 - o *: 0 atau lebih kali
 - o +: 1 atau lebih kali
 - o ?: 0 atau 1 kali
 - o {n}: Persis n kali
 - o {n,}: Setidaknya n kali
 - o {n,m}: Setidaknya n kali, maksimal m kali

Contoh Penggunaan Regex:

1. Mencocokkan pola

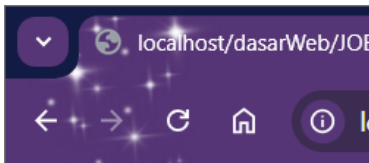
```
$pattern = '/hello/';  
$text = 'Hello, world!';  
if (preg_match($pattern, $text)) {  
    echo "Cocokkan ditemukan!";  
} else {  
    echo "Tidak ada cocokan!";  
}
```

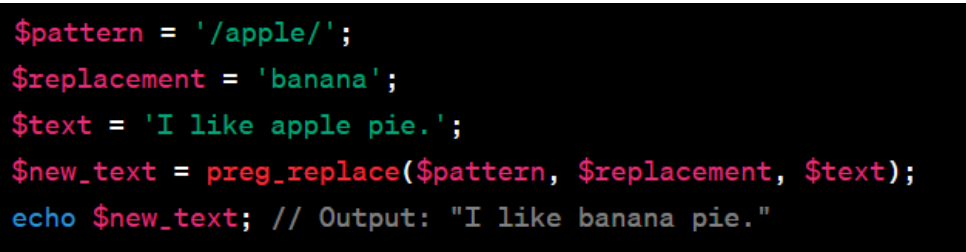
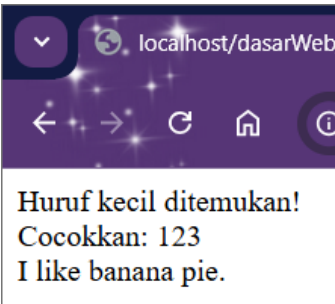
2. Mencocokkan karakter set

```
$pattern = '/[0-9]/'; // Cocokkan satu digit.  
$text = 'There are 3 apples.';  
if (preg_match($pattern, $text)) {  
    echo "Digit ditemukan!";  
} else {  
    echo "Tidak ada digit!";  
}
```

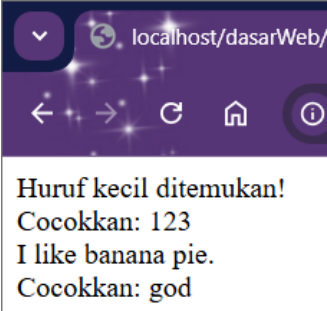
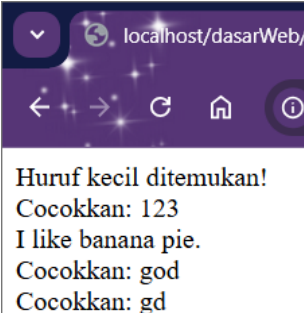
Praktikum 5 : Penggunaan Regex Pada PHP

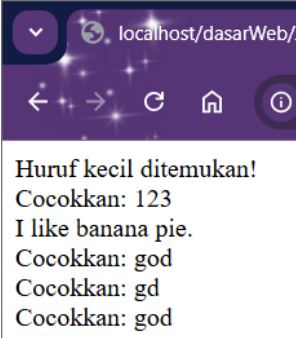
Langkah-langkah Praktikum :

Langkah	Keterangan
1	Buat satu file baru bernama <code>regex.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>regex.php</code>
2	<pre> \$pattern = '/[a-z]/'; // Cocokkan huruf kecil. \$text = 'This is a Sample Text.'; if (preg_match(\$pattern, \$text)) { echo "Huruf kecil ditemukan!"; } else { echo "Tidak ada huruf kecil!"; } </pre>
3	Simpan file , kemudian buka browser dan jalankan/refresh <code>localhost/dasarWeb/regex.php</code>
4	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.1) Jawab:  Huruf kecil ditemukan! Penjelasan: Kode PHP tersebut berfungsi untuk memeriksa apakah sebuah teks mengandung huruf kecil dengan bantuan regular expression (regex). Pola <code>/[a-z]/</code> menunjukkan pencarian huruf kecil dari a sampai z. Fungsi <code>preg_match()</code> digunakan untuk mencocokkan pola tersebut dengan isi variabel <code>\$text</code>, yaitu "This is a Sample Text.". Jika ditemukan huruf kecil, program akan menampilkan pesan "Huruf kecil ditemukan!", sedangkan jika tidak ada huruf kecil maka akan muncul pesan "Tidak ada huruf kecil!". Karena pada teks contoh terdapat huruf kecil seperti h, i, dan s, maka output yang dihasilkan adalah "Huruf kecil ditemukan!".
5	Ketikkan kode tambahan pada langkah 6 di dalam <code>regex.php</code>
6	<pre> \$pattern = '/[0-9]+/'; // Cocokkan satu atau lebih digit. \$text = 'There are 123 apples.'; if (preg_match(\$pattern, \$text, \$matches)) { echo "Cocokkan: " . \$matches[0]; } else { echo "Tidak ada yang cocok!"; } </pre>
7	Lengkapi kode program di atas sehingga hasilnya rapi. Simpan file , kemudian buka browser dan jalankan/refresh <code>localhost/dasarWeb/regex.php</code>

8	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.2) Jawab:  Penjelasan: Pada bagian kode baru ditambahkan pola regex <code>'/[0-9]+/</code> yang digunakan untuk mencari angka di dalam string. Regex ini berarti mencocokkan satu atau lebih digit angka (0–9) secara berurutan. Variabel <code>\$text</code> diisi dengan kalimat "There are 123 apples.", lalu fungsi <code>preg_match()</code> dijalankan dengan parameter tambahan <code>\$matches</code>. Jika pola cocok ditemukan, maka hasil pencocokan pertama akan disimpan di dalam array <code>\$matches</code>, dan program menampilkan angka tersebut melalui <code>echo "Cocokkan: " . \$matches[0];</code>. Karena string memang mengandung angka 123, maka output yang dihasilkan adalah <i>"Cocokkan: 123"</i>. Jika tidak ada angka, maka program akan menampilkan pesan <i>"Tidak ada yang cocok!"</i>.
9	Ketikkan kode tambahan pada langkah 10 di dalam <code>regex.php</code>
10	 <pre> \$pattern = '/apple/'; \$replacement = 'banana'; \$text = 'I like apple pie.'; \$new_text = preg_replace(\$pattern, \$replacement, \$text); echo \$new_text; // Output: "I like banana pie." </pre>
11	Lengkapi kode program di atas sehingga hasilnya rapi. Simpan file , kemudian buka browser dan jalankan/refresh <code>localhost/dasarWeb/regex.php</code>
12	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.3) Jawab:  Penjelasan: Pada bagian kode baru ditambahkan penggunaan fungsi <code>preg_replace()</code> untuk melakukan pencarian dan penggantian teks berdasarkan pola regex. Pola yang digunakan adalah <code>/apple/</code>, yang berarti mencari kata <i>apple</i> dalam string. Variabel <code>\$replacement</code> diisi dengan kata <i>banana</i>, sedangkan <code>\$text</code> berisi kalimat "I like apple pie.". Fungsi <code>preg_replace(\$pattern, \$replacement, \$text)</code> akan mencari kecocokan pola pada string, lalu

	menggantinya dengan nilai pengganti. Hasilnya disimpan dalam variabel <code>\$new_text</code> . Karena kata <i>apple</i> ditemukan di dalam string, maka teks tersebut diganti menjadi <i>banana</i> , sehingga output akhir yang ditampilkan adalah “I like banana pie.” Dengan demikian, bagian tambahan kode ini menunjukkan cara menggunakan regex tidak hanya untuk mendeteksi, tetapi juga mengubah isi string sesuai pola yang ditentukan.
13	Ketikkan kode tambahan pada langkah 14 di dalam <code>regex.php</code>

14	<pre> \$pattern = '/go*d/'; // Cocokkan "god", "good", "goood", dll. \$text = 'god is good.'; if (preg_match(\$pattern, \$text, \$matches)) { echo "Cocokkan: " . \$matches[0]; } else { echo "Tidak ada yang cocok!"; } </pre>
15	Lengkapi kode program di atas sehingga hasilnya rapi. Simpan file , kemudian buka browser dan jalankan/refresh localhost/dasarWeb/regex.php
16	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.4) Jawab:  Penjelasan: Pada bagian kode baru ditambahkan pola regex '/go*d/'. Pola ini berarti mencari teks yang diawali dengan huruf g, diikuti huruf o yang boleh muncul nol kali atau lebih (*), lalu diakhiri dengan huruf d. Variabel \$text diisi dengan kalimat "god is good.". Fungsi preg_match() kemudian mencari kecocokan berdasarkan pola tersebut. Karena dalam string terdapat kata "god" dan "good", regex berhasil menemukan kecocokan, dan hasil pertama disimpan dalam \$matches[0]. Output yang ditampilkan adalah "Cocokkan: god". Singkatnya, tambahan kode ini memperlihatkan penggunaan simbol * dalam regex untuk mencocokkan huruf atau karakter yang boleh muncul berkali-kali (termasuk nol kali).
17	Soal 5.5: Buatlah script langkah 14 dengan variable pattern yang di ubah menggunakan '?' (0 atau 1 kali). Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.5) Jawab: 

	Penjelasan: • Regex <code>go?d</code> akan mencocokkan kata "gd" (tanpa huruf o) atau "god" (dengan satu o), tetapi tidak akan mencocokkan "good" karena mengandung dua huruf o. • Pada contoh teks "gd and god and good.", hasil yang dicocokkan pertama kali adalah "gd". • Jadi penggunaan <code>?</code> membatasi kemunculan karakter menjadi opsional, maksimal 1 kali, berbeda dengan <code>*</code> yang memperbolehkan kemunculan karakter berkali-kali. • Jadi, simbol <code>?</code> membuat pola lebih ketat karena hanya menerima 0 atau 1 kemunculan karakter, bukan lebih.
18	Soal 5.6: Buatlah script langkah 14 dengan variable pattern yang di ubah menggunakan <code>{n,m}</code>. Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 5.6) Jawab:  Penjelasan: • Pola <code>/go{1,2}d/</code> akan mencocokkan kata "god" (1 huruf o) dan "good" (2 huruf o), tetapi tidak akan mencocokkan "gd" (tanpa o) maupun "goood" (3 o). • Fungsi <code>{n,m}</code> pada regex dipakai untuk menentukan berapa kali sebuah karakter boleh muncul. • Berbeda dengan <code>*</code> (nol atau lebih) atau <code>?</code> (nol atau satu), <code>{n,m}</code> memberikan kontrol lebih spesifik tentang jumlah kemunculan karakter. • Jadi, penggunaan <code>{n,m}</code> membuat pencocokan lebih terarah karena kita bisa mengatur batas minimal dan maksimal jumlah kemunculan karakter tertentu.

Praktikum 6 : Form Lanjut

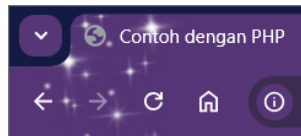
Langkah-langkah Praktikum :

Langkah	Keterangan
1	Buat satu file baru bernama <code>form_lanjut.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>form_lanjut.php</code>

2	<pre> <!DOCTYPE html> <html> <head> <title>Contoh Form dengan PHP</title> </head> <body> <h2>Form Contoh</h2> <form method="POST" action="proses_lanjut.php"> <label for="buah">Pilih Buah:</label> <select name="buah" id="buah"> <option value="apel">Apel</option> <option value="pisang">Pisang</option> <option value="mangga">Mangga</option> <option value="jeruk">Jeruk</option> </select>
 <label>Pilih Warna Favorit:</label>
 <input type="checkbox" name="warna[]" value="merah"> Merah
 <input type="checkbox" name="warna[]" value="biru"> Biru
 <input type="checkbox" name="warna[]" value="hijau"> Hijau

 <label>Pilih Jenis Kelamin:</label>
 <input type="radio" name="jenis_kelamin" value="laki-laki"> Laki-laki
 <input type="radio" name="jenis_kelamin" value="perempuan"> Perempuan

 <input type="submit" value="Submit"> </form> </body> </html> ' if (\$_SERVER["REQUEST_METHOD"] == "POST") { \$selectedBuah = \$_POST['buah']; if (isset(\$_POST['warna'])) { \$selectedWarna = \$_POST['warna']; } else { \$selectedWarna = []; } \$selectedJenisKelamin = \$_POST['jenis_kelamin']; echo "Anda memilih buah: " . \$selectedBuah . "
"; if (!empty(\$selectedWarna)) { echo "Warna favorit Anda: " . implode(", ", \$selectedWarna) . "
"; } else { echo "Anda tidak memilih warna favorit.
"; } echo "Jenis kelamin Anda: " . \$selectedJenisKelamin; } </pre>
3	Simpan file , kemudian buka browser dan jalankan/refresh localhost/dasarWeb/form_lanjut.php
4	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 6.1) Jawab: - Tampilan awal



Form Contoh

Pilih Buah :

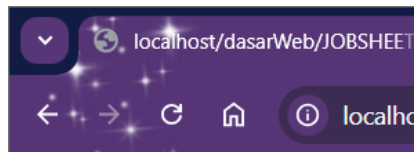
Pilih Warna Favorit:

- ☒ Merah
☐ Biru
☐ Hijau

Pilih Jenis Kelamin:

- ☐ Laki-laki
☒ Perempuan

- Tampilan setelah submit



Anda memilih buah: mangga
 Warna favorit Anda: merah
 Jenis kelamin Anda: perempuan

- Penjelasan:

- **Form HTML:**

- Dropdown <select> untuk memilih satu buah.
- Checkbox <input type="checkbox"> untuk memilih satu atau lebih warna favorit.
- Radio button <input type="radio"> untuk memilih jenis kelamin (satu pilihan saja).
- Tombol submit untuk mengirim data.

- **Proses PHP:**

- Mengecek apakah request dikirim dengan metode POST.
- Menyimpan dan menampilkan pilihan buah.
- Mengecek apakah ada warna favorit yang dipilih:
 - Jika ada, data ditampilkan dalam bentuk string dipisahkan koma.
 - Jika tidak, muncul pesan bahwa tidak memilih warna favorit.
- Menyimpan dan menampilkan jenis kelamin yang dipilih.

- **Output:**

- Menampilkan kembali pilihan buah.

	➤ Menampilkan warna favorit (bisa lebih dari satu). ➤ Menampilkan jenis kelamin.
5	Buat satu file baru bernama <code>form_ajax.php</code> di dalam folder <code>dasarWeb</code>. Ketikkan kode pada langkah 2 di dalam <code>form_ajax.php</code> <pre> <!DOCTYPE html> <html> <head> <title>Contoh Form dengan PHP dan jQuery</title> <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> </head> <body> <h2>Form Contoh</h2> <form id="myForm"> <label for="buah">Pilih Buah:</label> <select name="buah" id="buah"> <option value="apel">Apel</option> <option value="pisang">Pisang</option> <option value="mangga">Mangga</option> <option value="jeruk">Jeruk</option> </select>
 <label>Pilih Warna Favorit:</label>
 <input type="checkbox" name="warna[]" value="merah"> Merah
 <input type="checkbox" name="warna[]" value="biru"> Biru
 <input type="checkbox" name="warna[]" value="hijau"> Hijau

 <label>Pilih Jenis Kelamin:</label>
 <input type="radio" name="jenis_kelamin" value="laki-laki"> Laki-laki
 <input type="radio" name="jenis_kelamin" value="perempuan"> Perempuan

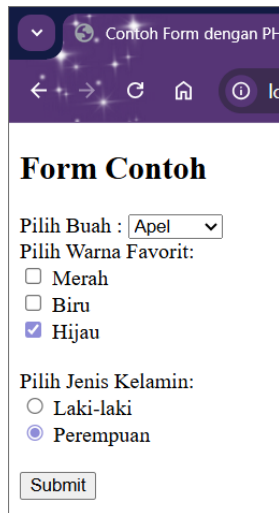
 <input type="submit" value="Submit"> </form> <div id="hasil"> <!-- Hasil akan ditampilkan di sini --> </div> <script> \$(document).ready(function () { \$("#myForm").submit(function (e) { e.preventDefault(); // Mencegah pengiriman form secara default // Mengumpulkan data form var formData = \$("#myForm").serialize(); // Kirim data ke server PHP \$.ajax({ url: "proses_lanjut.php", // Ganti dengan nama file PHP yang sesuai type: "POST", data: formData, success: function (response) { // Tampilkan hasil dari server di div "hasil" \$("#hasil").html(response); } }); }); }); </script> </body> </html> </pre>

6

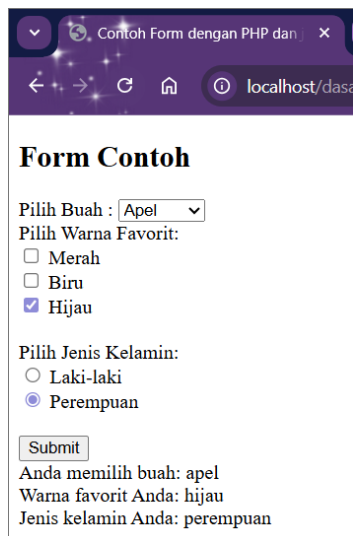
Simpan file , kemudian buka browser dan jalankan/refresh localhost/dasarWeb/form_ajax.php. Catat di sini apa yang anda amati dari penambahan kode program di atas.
(soal no 6.2)

Jawab:

- Tampilan awal



- Tampilan setelah submit



- **Penjelasan:**

Kode program di atas merupakan contoh penerapan form HTML yang dipadukan dengan jQuery dan AJAX untuk mengirim data ke server secara dinamis. Pada bagian form, pengguna dapat memilih buah melalui dropdown, memilih warna favorit dengan checkbox, serta memilih jenis kelamin dengan radio button. Biasanya, saat tombol submit ditekan, form akan dikirim ke server dan halaman akan dimuat ulang. Namun, di sini digunakan fungsi `e.preventDefault()` agar pengiriman data tidak dilakukan secara default. Data yang diisi pengguna dikumpulkan menggunakan `serialize()` sehingga seluruh input form bisa dijadikan satu paket data. Selanjutnya, data tersebut dikirim ke file `proses_lanjut.php` melalui AJAX dengan metode POST. Hasil balasan dari server kemudian ditampilkan langsung di dalam elemen `<div id="hasil">` tanpa perlu me-refresh halaman.

Praktikum 7 : Validasi Form

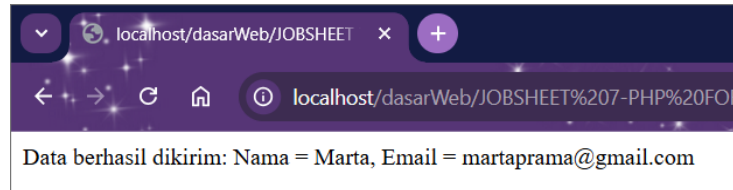
Langkah-langkah Praktikum :

Langkah	Keterangan
1	Buat satu file baru bernama <code>form_validasi.php</code> di dalam folder <code>dasarWeb</code> . Ketikkan kode pada langkah 2 di dalam <code>form_validasi.php</code>
2	<pre> <!DOCTYPE html> <html> <head> <title>Form Input dengan Validasi</title> </head> <body> <h1>Form Input dengan Validasi</h1> <form method="post" action="proses_validasi.php"> <label for="nama">Nama:</label> <input type="text" id="nama" name="nama">
 <label for="email">Email:</label> <input type="text" id="email" name="email">
 <input type="submit" value="Submit"> </form> </body> </html> </pre> <pre> <?php if (\$_SERVER["REQUEST_METHOD"] == "POST") { \$nama = \$_POST["nama"]; \$email = \$_POST["email"]; \$errors = array(); // Validasi Nama if (empty(\$nama)) { \$errors[] = "Nama harus diisi."; } // Validasi Email if (empty(\$email)) { \$errors[] = "Email harus diisi."; } elseif (filter_var(\$email, FILTER_VALIDATE_EMAIL)) { \$errors[] = "Format email tidak valid."; } // Jika ada kesalahan validasi if (empty(\$errors)) { (foreach (\$errors as \$error) { echo \$error . "
"; } } else { // Lanjutkan dengan pemrosesan data jika semua validasi berhasil // Misalnya, menyimpan data ke database atau mengirim email echo "Data berhasil dikirim: Nama = \$nama, Email = \$email"; } } ?> </pre>
3	Simpan file , kemudian buka browser dan jalankan/refresh <code>localhost/dasarWeb/form_validasi.php</code>
4	Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 7.1) Jawab:

- Tampilan awal



- Tampilan setelah submit



- Penjelasan:

- **Form HTML**

- Form memiliki dua input: nama dan email.
 - Data form dikirim menggunakan metode **POST** ke file `proses_validasi.php` untuk diproses.
 - Tombol **Submit** digunakan untuk mengirim data.

- **Skrip PHP Validasi**

- PHP akan berjalan hanya jika form dikirim (`$_SERVER["REQUEST_METHOD"] == "POST"`).
 - Variabel `$nama` dan `$email` diambil dari input form (`$_POST`).
 - Array `$errors` digunakan untuk menyimpan pesan error jika ada kesalahan validasi.

- **Validasi Nama**

- Dicek apakah field nama kosong.
 - Jika kosong, maka pesan error "**Nama harus diisi.**" akan dimasukkan ke dalam array `$errors`.

- **Validasi Email**

- Dicek apakah field email kosong.
 - Jika kosong, akan muncul pesan "**Email harus diisi.**"
 - Jika terisi, tetapi format tidak sesuai (dicek dengan `filter_var` dan `FILTER_VALIDATE_EMAIL`), maka akan muncul pesan "**Format email tidak valid.**"

- **Menampilkan Hasil Validasi**

	➤ Jika ada error, semua pesan error ditampilkan dengan perulangan foreach. ➤ Jika tidak ada error, data akan diproses lebih lanjut, misalnya disimpan ke database atau dikirim melalui email. ➤ Pada contoh ini, program hanya menampilkan pesan konfirmasi: Data berhasil dikirim: Nama = ..., Email = ...
5	Kembangkan file bernama form_validasi.php di dalam folder dasarWeb. Ketikkan kode pada langkah 6 di dalam form_validasi.php
6	<pre> <!DOCTYPE html> <html> <head> <title>Form Input dengan Validasi</title> <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> </head> <body> <h1>Form Input dengan Validasi</h1> <form id="myForm" method="post" action="proses_validasi.php"> <label for="nama">Nama:</label> <input type="text" id="nama" name="nama">
 <label for="email">Email:</label> <input type="text" id="email" name="email">
 <input type="submit" value="Submit"> </form> <script> \$(document).ready(function() { \$("#myForm").submit(function(event) { var nama = \$("#nama").val(); var email = \$("#email").val(); var valid = true; if (nama === "") { \$("#nama-error").text("Nama harus diisi."); valid = false; } else { \$("#nama-error").text(""); } if (email === "") { \$("#email-error").text("Email harus diisi."); valid = false; } else { \$("#email-error").text(""); } if (valid) { (!event.preventDefault()); // Menghentikan pengiriman form jika validasi gagal } }); }); </script> </body> </html> </pre>
7	Simpan file , kemudian buka browser dan jalankan/refresh localhost/dasarWeb/form_validasi.php

8

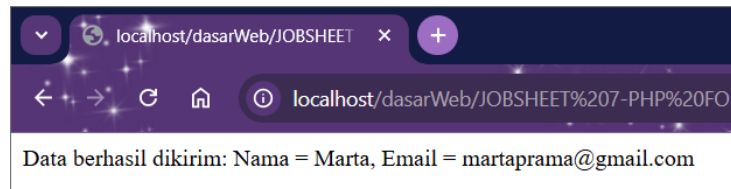
Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 7.2)

Jawab:

- Tampilan awal



- Tampilan setelah submit



- **Penjelasan:**

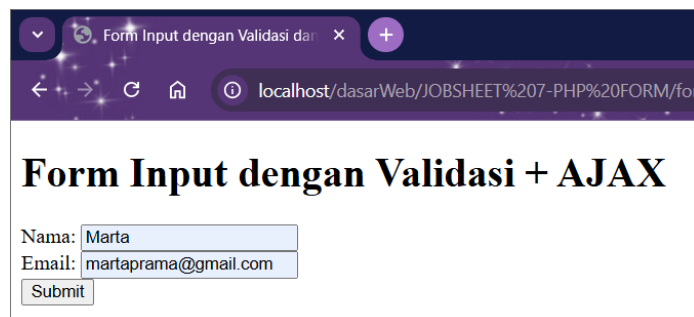
Perubahan kode tersebut menunjukkan bahwa validasi input form kini dilakukan di sisi **client** menggunakan jQuery sebelum data dikirim ke server. Dengan adanya script ini, ketika pengguna menekan tombol submit, sistem terlebih dahulu mengecek apakah field **Nama** dan **Email** sudah diisi. Jika ada yang kosong, pesan error langsung ditampilkan di bawah field terkait (misalnya "Nama harus diisi" atau "Email harus diisi"), dan pengiriman form dihentikan menggunakan `event.preventDefault()`. Hal ini membuat pengguna mendapat umpan balik secara langsung tanpa perlu memuat ulang halaman, sehingga data yang dikirim ke server lebih terjamin kebenarannya dan pengalaman pengguna menjadi lebih interaktif.

9

Soal 7.3: Buatlah script langkah 6 dengan menggunakan ajax. Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 7.3)

Jawab:

- Tampilan awal



- Tampilan setelah submit

- **Penjelasan:**

- **Form tidak lagi melakukan reload halaman** saat tombol submit ditekan.
→ Data dikirim ke server lewat AJAX, hasil balasan server langsung ditampilkan di `<div id="result">`.
- **Validasi tetap dilakukan di sisi client** menggunakan jQuery sebelum data dikirim.
→ Jika ada field kosong, error muncul di bawah input tanpa mengirim request ke server.
- **Respon server ditangani langsung di halaman.**
→ Misalnya jika `proses_validasi.php` mengembalikan pesan "Data berhasil disimpan", pesan tersebut langsung muncul di area `#result`.
- **User experience lebih baik** karena tidak ada refresh halaman, lebih interaktif, dan cepat.

10

Soal 7.4: Tambahkan validasi untuk password dengan ketentuan minimal 8 karakter dengan menggunakan jQuery dan PHP tambahkan pada langkah 9. Catat di sini apa yang anda amati dari penambahan kode program di atas. (soal no 7.4)

Jawab:

- Tampilan awal

- Tampilan setelah submit

	- Penjelasan: • Di sisi jQuery (client-side): ➤ Form langsung memberi peringatan kalau password kurang dari 8 karakter tanpa harus reload halaman. ➤ User bisa memperbaiki input sebelum dikirim ke server → lebih cepat dan interaktif. • Di sisi PHP (server-side): ➤ Validasi tetap dilakukan agar data yang masuk ke server tetap aman, meskipun user mencoba memanipulasi kode JavaScript. ➤ Jika password kosong atau kurang dari 8 karakter, server memberikan pesan error. • Kombinasi client-side + server-side validasi: ➤ Memberikan pengalaman pengguna yang baik (cepat karena validasi awal dilakukan di browser). ➤ Tetap menjaga keamanan data karena validasi akhir tetap dilakukan di server. • UX lebih baik & keamanan lebih terjamin. ➤ User merasa nyaman karena ada feedback langsung. ➤ Developer tetap aman dari input yang tidak valid.
--	---

